

Runtime Scriptable Edit

Setup Guide and API Reference

Baran Koc

Version 1.0 – Unity 6000.0.68f1 or newer – Editor-only

Table of Contents

1. What the package does	1
2. Installation	2
3. Setup, step by step	2
4. The demo scene	3
5. Apply, Perma Save and Revert.....	3
6. Writing code that supports live tuning.....	4
7. How it works	5
8. API reference	6
9. Limitations	8
10. Troubleshooting.....	8

1. What the package does

Unity treats ScriptableObject assets as read-only-ish during Play Mode: you *can* edit them, but the change is written straight to the asset on disk, and Unity restores the pre-play values when you leave Play Mode. Tuning a jump height by feel therefore means either losing the value on exit, or writing it down and typing it back in afterwards.

Runtime Scriptable Edit removes that choice:

- On entering Play Mode it makes an in-memory **runtime copy** of every asset you nominated, plus a **snapshot** used as the revert point.
- It then rewrites the references held by your scene objects to point at the runtime copy, so your game reads the copy rather than the asset.
- You edit the copy through a dedicated window while the game runs.
- On exit, anything not explicitly **Perma Saved** is rolled back.

Your original assets are never modified unless you ask for it.

2. Installation

From the Asset Store

Import the package. It installs to Assets/RuntimeScriptableEdit/.

Manually

Copy the RuntimeScriptableEdit folder anywhere inside your project's Assets folder.

It does **not** need to live in an Editor folder. The package ships three assembly definitions and the editor assembly is already restricted to the Editor platform:

Assembly	Location	Platforms
BaranKoc.RuntimeScriptableEdit	Runtime/	All
BaranKoc.RuntimeScriptableEdit.Editor	Editor/	Editor only
BaranKoc.RuntimeScriptableEdit.Demo	Demo/	All

Only RuntimeScriptableEditProfile lives in the runtime assembly, so referencing a profile from your own gameplay code will not break a player build. All behaviour is editor-side.

Removing the demo

The Demo folder is self-contained. Delete it and nothing else stops working.

3. Setup, step by step

1 — Create a profile.

Assets > Create > Runtime Scriptable Edit > Profile

A profile is just a list of the assets you want to tune. Most projects need one; large projects often keep several (CombatTuning, MovementTuning) and switch between them.

2 — Populate Tunable Assets.

Select the profile, press + on the Tunable Assets list, then fill the new row either by dragging a ScriptableObject onto it or by clicking the picker button at the right edge of the field and choosing one. There is no interface to implement and no base class to inherit — the package works with the config assets you already have.

Note that Unity's + copies the last row rather than adding an empty one, so a new row usually arrives already holding the previous asset. Just pick the asset you actually want over the top of it. Empty and duplicated rows are harmless — they are skipped, with a warning naming the profile, when you enter Play Mode.

3 — Open the window.

Tools > RuntimeScriptableEdit > Runtime Scriptable Edit Window

4 — Assign the active profile.

In Edit Mode the window shows an Active Profile field. Drag your profile onto it, or click the picker button at the right edge of the field and select the profile from the list.

The choice is stored in EditorPrefs, so it is per-user rather than committed to the project. Two people on the same project can each have a different profile active.

5 — Configure confirmations (optional).

Below the profile field are three toggles controlling whether Apply, Perma Save and Revert ask for confirmation. All default to on. Turn them off once the workflow is familiar.

6 — Enter Play Mode.

The window switches to its tuning view and lists an inspector for each tunable asset.

4. The demo scene

Demo/Scenes/RuntimeScriptableEditDemo.unity

A capsule patrols left and right and bounces, driven entirely by DemoMovementConfig. An on-screen readout prints the values being read on the current frame.

Open the scene, open the window, and set Active Profile to Demo/Data/DemoRuntimeScriptableEditProfile.asset — it ships already populated, so that one assignment is the whole setup. The active profile itself lives in EditorPrefs rather than in the package, so it cannot arrive pre-selected on your machine.

Then press Play and drag Bounce Height or Move Speed. The readout and the capsule change together — that pairing is the proof that your scene is reading the runtime copy rather than the asset.

Body Color is the fastest thing to try: the capsule recolours the instant the swatch changes.

The demo moves itself instead of reading input, so it behaves the same whether your project uses the legacy Input Manager, the Input System package, or both.

5. Apply, Perma Save and Revert

Three operations, available per-asset and for all assets at once.

Operation	Writes to the asset on disk	Survives leaving Play Mode	Moves the revert point
Temporary Apply	Yes	No — rolled back on exit	No
Perma Save	Yes	Yes	Yes
Revert	Yes	n/a	No — restores it

Temporary Apply exists for systems that read the asset directly rather than through a patched reference — an asset loaded through `Resources.Load` or `Addressables`, for example. Pushing the values onto the real asset makes those systems see them too, and the change is still undone on exit.

Perma Save is the one that keeps your work. It writes the runtime values to the asset *and* updates the snapshot, so exiting Play Mode has nothing to roll back.

Revert restores the values captured at the last Perma Save, or at the moment Play Mode started if you have not Perma Saved.

On exiting Play Mode, every entry that has not been Perma Saved is reverted automatically. Perma Saved entries keep their values.

Because Temporary Apply and Revert both write to the asset on disk, a crash or a forced quit during Play Mode can leave an asset holding temporary values. If you are tuning something you cannot afford to lose, commit it to source control first.

6. Writing code that supports live tuning

One rule: **read config values at the point of use, not at startup.**

```
public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private MovementConfig config;

    // Works. config.moveSpeed is read on the frame it is used, so it picks up
    // whatever the tuning window last set.
    private void Update()
    {
        transform.position += transform.forward * config.moveSpeed * Time.deltaTime;
    }
}

public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private MovementConfig config;
```

```

private float moveSpeed;

// Does not work. The value was copied out of the asset before you could
edit it,
// and nothing will ever update the copy.
private void Awake()
{
    moveSpeed = config.moveSpeed;
}
}

```

The package redirects your component's **reference**. A value already copied into a local field is out of reach — there is no way for any tool to find it.

Caching the *reference itself* is fine and expected. Only caching the *values* breaks.

Supported field shapes

The reference replacer patches these on every MonoBehaviour in the loaded scenes (including inactive ones) and on every non-asset ScriptableObject in memory:

- a single field of a ScriptableObject type
- an array of a ScriptableObject type
- a List<T> where T is a ScriptableObject type

Public and private fields are both handled. readonly fields are skipped.

Nested ScriptableObjects work: a config that references another config gets patched too, because runtime copies are themselves scanned.

Objects spawned during Play Mode

A prefab instantiated after Play Mode began holds the original reference, not the patched one. Two options:

- press **Re-patch References** in the window, or
- call `RuntimeScriptableEditReferenceReplacer.RepatchSingleObject(obj)` from your spawn code.

7. How it works

Entering Play Mode

- RuntimeScriptableEditBootstrap hears EnteredPlayMode
- Registry builds, per tunable asset:
 - runtimeCopy ← the object your scene will actually read
 - revertSnapshot ← the values restored by Revert
- ReferenceReplacer walks every MonoBehaviour and in-memory

ScriptableObject and repoints matching fields at runtimeCopy, recording each change so it can be undone

Leaving Play Mode

- ReferenceReplacer puts every patched field back to the original asset
- Registry reverts every entry that was not Perma Saved
- Registry destroys the runtime copies and snapshots

Restoring the references before destroying the copies is what makes the package safe under **Fast Enter Play Mode** with *Reload Scene* disabled. In that configuration Unity does not rebuild the scene on exit, so fields left pointing at destroyed copies would throw on the next play session.

Patching happens on `EnteredPlayMode`, which fires after the first scene's `Awake` and `OnEnable` but before the first `Update`. This is the reason for the rule in section 6: anything read during `Awake` is read before the swap.

8. API reference

Everything except `RuntimeScriptableEditProfile` is in the Editor assembly. Wrap calls from your own runtime scripts in `#if UNITY_EDITOR`.

Namespace: `BaranKoc.RuntimeScriptableEdit`

RuntimeScriptableEditProfile : ScriptableObject

Runtime assembly. Safe to reference from gameplay code.

Member	Type	Description
<code>tunableAssets</code>	<code>List<ScriptableObject></code>	Assets that get a runtime copy
<code>autoRepatchOnInstantiate</code>	<code>bool</code>	Reserved for future automatic re-patching

RuntimeScriptableEditBootstrap

Static. Subscribes to Play Mode transitions via `[InitializeOnLoad]`.

Member	Description
<code>GetActiveProfile()</code>	Returns the active profile, or null
<code>SetActiveProfile(profile)</code>	Sets it; pass null to clear

RuntimeScriptableEditRegistry

Static. Owns the runtime copies.

Member	Description
IsInitialized	true once at least one entry exists
ActiveProfile	Profile the registry was built from
Entries	IReadOnlyDictionary<ScriptableObject, ScriptableEditEntry>, keyed by original asset
Initialize(profile)	Rebuilds the registry from a profile
GetEntry(asset)	Entry for an original asset, or null
GetRuntimeCopy(asset)	Runtime copy for an original asset, or null
IsOriginalAsset(asset)	Whether the asset is registered
ApplyAll() / PermaSaveAll() / RevertAll()	Bulk operations
RevertTemporaryChanges()	Reverts every entry not Perma Saved
Clear()	Destroys all copies and empties the registry

RuntimeScriptableEditRegistry.ScriptableEditEntry

Member	Description
originalAsset	The asset on disk
runtimeCopy	The copy your scene reads
revertSnapshot	Values used by Revert()
isPermaSaved	Whether the entry survives Play Mode exit
Apply()	Copies runtime values to the asset, without moving the revert point
PermaSave()	Copies runtime values to the asset and updates the revert point
Revert()	Restores the snapshot into the runtime copy and applies it
Cleanup()	Destroys the copy and snapshot

RuntimeScriptableEditReferenceReplacer

Member	Description
ReplaceAllReferences()	Scans all loaded objects and repoints matching fields
RepatchSingleObject(obj)	Patches one object — use for runtime-spawned objects
RestoreAllReferences()	Puts every patched field back to its original asset
ReplacementCount	Number of fields patched by the last run

9. Limitations

- **Editor-only.** There is no runtime tuning in a player build.
 - **Values read during Awake or OnEnable** of the first scene are read before patching.
 - **Values copied into local fields** cannot be tuned. See section 6.
 - **Dictionary and nested serialized-struct fields** are not patched; single fields, arrays and `List<T>` are.
 - **Assets loaded by path** (`Resources.Load`, `Addressables`) are not reached by reference patching. Use **Temporary Apply** to push values onto the real asset for those systems.
 - **The active profile is stored in EditorPrefs**, so it is per-user and not shared through source control.
 - **Temporary Apply and Revert write to disk.** A crash mid-session can leave temporary values in an asset.
-

10. Troubleshooting

The window says “not initialized” in Play Mode. No active profile, or the profile’s Tunable Assets list is empty. Both are set in Edit Mode; leave Play Mode, fix it, and re-enter.

Editing a value does nothing. Almost always the caching mistake in section 6. Check whether the value is read in Update or copied in Awake. If your system loads the asset by path rather than through a serialized field, use **Temporary Apply** instead.

A newly spawned object ignores the tuning. It was created after patching. Press **Repatch References**, or call `RepatchSingleObject` when you spawn it.

My changes disappeared when I left Play Mode. They were not Perma Saved. Only **Perma Save** survives; **Temporary Apply** is rolled back by design.

Values in the window look wrong after a crash. An interrupted session can leave temporarily applied values on the asset. Restore the asset from source control.